

ARRAIS KARIM - BARRIAL NOA - CARUELLE JOHAN - POULAIN LAURA

MSPR NTL-SYSTOOLBOX

Outil de supervision et d'audit
d'exploitation

Table des matières

I. Introduction.....	3
1. Contexte.....	3
2. Problématiques identifiées.....	3
3. Objectifs de l'outil.....	4
II. Analyse du besoin.....	5
III. Présentation de la solution.....	6
1. Choix technologiques.....	6
1. Architecture de l'outil.....	7
2. Infrastructure minimale de test.....	8
IV. Module 1 : Diagnostic.....	10
1. Objectifs et fonctionnalités.....	10
2. Fonctionnement.....	10
3. Développement du script.....	11
4. Vérifications réalisées avec exemples de sorties.....	15
5. Complications rencontrées et limites.....	17
V. Module 2 : Sauvegarde WMS.....	18
1. Objectifs et fonctionnalités.....	18
2. Processus de sauvegarde.....	18
3. Vérification et exemples de sorties.....	20
4. Sécurité et traçabilité.....	22
5. Complications rencontrées et limites.....	22
VI. Module 3 : Audit d'obsolescence.....	23
1. Méthodologie d'inventaire.....	25
2. Détection OS.....	26
3. Source EOL (éditeur).....	27
4. Classification des risques.....	27
5. Exemple de rapport.....	28
6. Complications rencontrées et limites.....	29
VII. Mise en place de l'infrastructure de test.....	31
1. Mise en place Active Directory et DNS.....	31
2. Mise en place serveur Linux de base de données.....	32
3. Machines clientes.....	33
a. Windows.....	33
b. Linux - Ubuntu Desktop (24.0).....	34
c. Vérification de l'ajout des machines au domaine.....	35
VIII. Axes d'amélioration futures.....	35
Conclusion.....	37
Summary.....	38

I. Introduction

1. Contexte

Nord Transit Logistics (NTL) est une PME spécialisée dans la logistique, implantée dans la région Hauts-de-France avec un siège social à Lille et trois entrepôts principaux : Lens (WH1), Valenciennes (WH2) et Arras (WH3).

L'entreprise évolue dans un secteur où la réactivité est essentielle, avec des amplitudes horaires larges (5h30 – 18h30) et une forte dépendance à l'outil informatique pour la gestion des flux de marchandises.

Le système d'information de NTL repose sur une infrastructure hybride : une vingtaine de machines virtuelles (Linux et Windows) hébergées au siège, assurant des services fondamentaux tels que l'Active Directory, le DNS et surtout le logiciel métier "Warehouse Management System" (WMS).

Ce WMS, basé sur une base de données MySQL, constitue le cœur du fonctionnement de la société ; son indisponibilité entraîne l'arrêt immédiat des réceptions et expéditions sur l'ensemble des sites.

2. Problématiques identifiées

L'audit du système actuel révèle plusieurs points faibles qui menacent la continuité d'activité de NTL :

- **Absence de résilience et de tests** : bien que des sauvegardes existent, elles ne sont pas régulièrement testées ni stockées sous forme de copies immuables, ce qui expose l'entreprise à une perte de données majeure en cas d'incident.
- **Supervision limitée** : les outils actuels (comme Zabbix) se limitent à un suivi technique de "bas niveau" (disponibilité des hôtes, espace disque) sans visibilité réelle sur la santé des services applicatifs ou des processus métiers comme les flux EDI ou la réplication de base de données.
- **Risque d'obsolescence** : le parc informatique, composé notamment de Windows Server dont les versions ne sont pas toujours précisées, nécessite un suivi rigoureux des cycles de fin de vie (EOL) pour garantir la sécurité et le support éditeur.

- **Contraintes opérationnelles fortes** : avec une équipe informatique réduite (un responsable, un administrateur itinérant, un technicien et un alternant), les interventions doivent être rapides, fiables et sont souvent réalisées sur des fenêtres de maintenance nocturnes très courtes.

3. Objectifs de l'outil

Pour répondre à ces enjeux, la Direction IT a mandaté le développement de **NTL-SysToolbox**. Cet utilitaire en ligne de commande (CLI) est destiné à automatiser et simplifier les tâches d'administration quotidiennes à travers trois priorités principales :

1. **Fiabiliser le diagnostic** : en offrant une sonde capable de vérifier instantanément la cohérence des services AD/DNS et l'état de santé (Uptime, CPU, RAM) des serveurs critiques sous Windows et Ubuntu.
2. **Sécuriser les sauvegardes** : automatiser les exports complets ou partiels (SQL et CSV) de la base WMS, pour garantir la traçabilité et l'intégrité des sauvegardes.
3. **Anticiper les risques d'obsolescence** : intégrer un module d'audit réseau capable d'identifier les systèmes d'exploitation présents et de les comparer à leurs informations de fin de vie (EOL), permettant une planification budgétaire et technique des mises à jour.

L'objectif final est de fournir à la DSI un outil robuste, portable et documenté, permettant une gestion plus proactive et sécurisée de l'infrastructure, tout en minimisant les risques d'interruption de service.

II. Analyse du besoin

Le fonctionnement actuel du système d'information de **Nord Transit Logistics** révèle que plusieurs tâches d'administration sont encore réalisées manuellement, comme la vérification des services réseau, les sauvegardes de la base de données WMS, ou encore le suivi de l'état des serveurs.

Ces opérations mobilisent du temps aux équipes et présentent un risque d'erreur liée à la manipulation humaine.

L'entreprise a donc exprimé le besoin d'une solution, en CLI, permettant d'automatiser ces tâches, tout en restant simple à utiliser et à maintenir. L'objectif est de faciliter le travail des techniciens en regroupant les fonctions de **diagnostic**, de **sauvegarde** et d'**audit** dans un même outil, sans passer par plusieurs logiciels distincts.

Il est également important de prendre en compte la nécessité de compatibilité entre les environnements Windows et Linux, afin de s'adapter et de pouvoir fonctionner sur l'ensemble de l'infrastructure de NTL.

De plus, la solution doit permettre une traçabilité complète, un enregistrement automatique de rapports, et une installation rapide pour pouvoir être déployée facilement sur l'ensemble des machines l'utilisant.

L'analyse de ces besoins a conduit au développement de **NTL-SysToolbox**, un outil en ligne de commande, pour automatiser, centraliser et fiabiliser les principales opérations d'administration système.

III. Présentation de la solution

L'outil *NTL-SysToolbox* a été conçu sous la forme d'un menu en ligne de commande interactif. L'application est lancée depuis un script, qui ouvre automatiquement le menu principal de l'outil, ce qui facilite son exécution.

Il permet d'exécuter 3 fonctionnalités (modules) de manière indépendante :

1. Le **diagnostic du bon fonctionnement** notamment : de l'Active Directory, du DNS, de la base de données, des informations et ressources systèmes - sur Windows Server et Ubuntu.
2. La **sauvegarde de la base métier** (WMS) au format **SQL**, avec une possibilité d'**export** de table au format **CSV**.
3. L'**audit d'obsolescence** des systèmes clients, permettant de réaliser un inventaire réseau (composants, versions des systèmes d'exploitations utilisés, date de fin de vie), disponible dans un rapport exploitable généré pendant l'audit.

1. Choix technologiques

Le développement de l'outil a été réalisé avec **Python** afin d'assurer sa portabilité entre des environnements Windows et Linux, avec une exécution plus légère.

Python propose également de nombreuses bibliothèques intégrés (tels que `os`, `subprocess`, ou `mysql.connector`), réduisant la complexité de développement de nombreuses fonctionnalités.

Ce choix permet de répondre à l'objectif de concevoir un **outil universel**, exécutable à la fois depuis un poste **Windows** et depuis une machine **Linux**, sans contraintes de compilation ou d'installation complexe. Cela permet à l'outil d'être facilement déployé, et ce dans tous types d'environnements.

Enfin, l'outil communique avec une base **MySQL**, utilisée pour la gestion et la sauvegarde des données. Ce choix repose principalement sur la facilité d'intégration avec le module `mysql.connector` de Python, qui permet de se connecter avec la base de données.

1. Architecture de l'outil

Organisation des modules

Chaque module est développé dans un fichier Python distinct (**diagnostic.py**, **sauvegarde.py**, **audit.py**) et est appelé via un menu principal interactif, géré par le script **main.py**. Cela permet à l'administrateur de lancer les diagnostics ou sauvegardes sans avoir à modifier le code source.

Gestion de la configuration

La configuration du projet repose sur un fichier **.env**, contenant les paramètres de connexion utiles pour l'exécution des modules (adresses IP, identifiants, informations MySQL).

Ces variables sont chargées automatiquement par le script grâce au module **python-dotenv**, ce qui permet d'adapter l'outil à différents environnements sans modifier le code source.

Pour simplifier la mise en place sur les postes de test, une version préconfigurée du fichier **.env** a été intégrée au projet. En situation réelle, ce fichier ne devrait pas être versionné pour des raisons de sécurité : seulement une copie d'exemple (**.env.exemple**) devrait être partagée pour servir de modèle lors des déploiements.

Gestion des secrets

Aucune donnée de connexion (identifiants, mots de passe) n'est stockée en clair dans le code : toutes les informations sensibles sont centralisées dans le fichier **.env**, accessible uniquement par l'administrateur.

Cela respecte les bonnes pratiques de sécurité et d'isolation des secrets. De plus, les accès **MySQL** et **WinRM** sont limités à des comptes dédiés à privilèges restreints.

Gestion des logs et retours

Chaque module affiche ses résultats directement dans le terminal. Les modules 2 et 3 génèrent en parallèle des fichiers de sortie dans les dossiers :

- **backups/** : fichiers .sql et exports .csv
- **outputs/** : rapports d'obsolescence

Les sorties sont **horodatées** afin d'assurer une traçabilité complète des actions. Les erreurs et exceptions sont également gérées par des messages explicites, facilitant le debug et la maintenance de l'outil.

2. Infrastructure minimale de test

L'infrastructure a été choisie pour permettre aux différentes fonctionnalités de l'outil d'être testées, le tout en restant adaptée à un environnement de test.

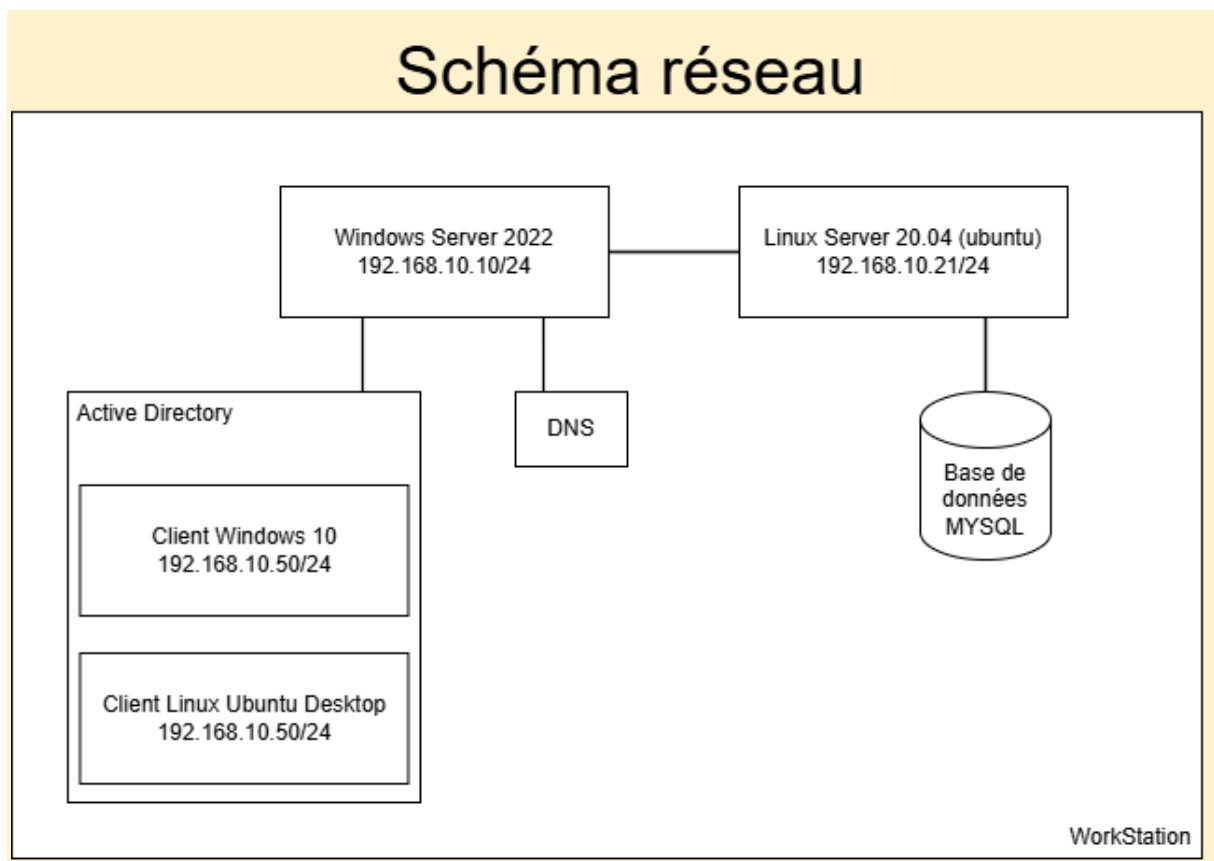
Nous avons sélectionné les machines principales sur lesquelles est basé le fonctionnement du système d'information de NTL.

Rôle	Machine	Adresse IP	Module(s) à tester
Contrôleur de domaine principal	Windows Server 2022	192.168.10.10	• Module 1: test du diagnostic des services AD/DNS et de celui des ressources (pour Windows Server)
Serveur de base de données	Linux Ubuntu Server 20.04	192.168.10.21	• Module 1: test du diagnostic de la base de données et celui des ressources (pour Ubuntu) • Module 2: test de sauvegarde de la base de données et d'export CSV
Client Windows	Windows 10	192.168.10.50	• Module 3: test de l'audit d'obsolescence pour Windows
Client Linux	Linux Ubuntu Desktop	192.168.10.50	• Module 3: test de l'audit d'obsolescence pour Linux

Ces machines constituent l'infrastructure **minimale** pour réaliser tous les tests de bon fonctionnement des modules de l'outil. Les autres serveurs peuvent être exclus de l'infrastructure de test.

Nous avons par exemple décidé de ne pas intégrer les machines telles que le serveur de téléphonie ou le serveur de supervision, n'étant pas nécessaires pour réaliser les tests fonctionnels.

Un seul contrôleur de domaine principal a été conservé : la haute disponibilité assurée par un second serveur n'est pas indispensable dans notre environnement de test (l'objectif étant de vérifier le bon fonctionnement des services Active Directory et DNS).



IV. Module 1 : Diagnostic

1. Objectifs et fonctionnalités

Le module de diagnostic de l'outil NTL-SysToolbox a pour objectif principal de permettre une **supervision centralisée** et **automatisée** des serveurs qui composent le réseau de l'entreprise NTL. Il permet d'avoir une vue globale sur le bon fonctionnement de l'infrastructure.

Le module intègre plusieurs fonctionnalités :

- Vérification de la disponibilité réseau et des services Active Directory et DNS
- Vérification de la base MySQL WMS : connectivité et accessibilité du service
- Récupération d'informations système (OS, uptime, charge CPU, mémoire utilisée, espace disque) pour Ubuntu et Windows Server

L'objectif de ce module est donc d'identifier rapidement les potentiels problèmes avant qu'ils impactent la production (en cas de dysfonctionnement de l'Active Directory, ou de base de données inaccessible ou corrompue par exemple).

2. Fonctionnement

Le module de **diagnostic** est lancé depuis le **menu principal CLI** de *NTL-SysToolbox*, en sélectionnant l'option 1.

Après sélection, le script **diagnostic.py** s'exécute automatiquement et affiche le résultat directement dans le terminal.

```
=====
[ NTL-SysToolbox ]
=====
Date : 2025-12-19 16:19:39

Sélectionnez un module à exécuter :
1. Diagnostic
2. Sauvegarde WMS
3. Audit d'obsolescence
0. Quitter

Votre choix : █
```

Une fois le script terminé, il est possible de revenir à l'écran principal et de choisir un autre module à lancer (cela est également prévu pour les modules 2 et 3). Cela permet de lancer tous les modules souhaités sans avoir besoin de relancer l'intégralité de l'outil à chaque fois.

```
=== Diagnostic terminé ===  
[OK] Module Diagnostic terminé.  
Appuyez sur Entrée pour revenir au menu...
```

L'objectif de la structure de l'affichage est d'être lisible, en indiquant les différentes étapes au fur et à mesure dans la sortie du script, afin que les utilisateurs puissent suivre facilement la progression du diagnostic en temps réel.

3. Développement du script

Le code est structuré sous la forme de blocs indépendants. Le script importe plusieurs bibliothèques nécessaires pour le diagnostic (**mysql.connector**, **dns.resolver**, **winrm** par exemple).

Il charge ensuite les variables d'environnement depuis le fichier **.env** grâce à la bibliothèque **dotenv** (fonction **load_dotenv()**), ce qui permet de ne pas stocker les informations sensibles (identifiants, mots de passe) directement dans le code source, mais dans un fichier séparé. Cela limite ainsi les risques de sécurité.

```
load_dotenv()  
print("Chargement du .env ...")  
print("Windows server :", os.getenv("WIN_HOST"))  
  
SERVER_DNS = os.getenv("SERVER_DNS", "192.168.10.10")  
SERVER_AD = os.getenv("SERVER_DNS", "192.168.10.10")  
SERVER_MYSQL = os.getenv("MYSQL_HOST", "192.168.10.21")  
MYSQL_USER = os.getenv("MYSQL_USER")  
MYSQL_PASSWORD = os.getenv("MYSQL_PASSWORD")  
MYSQL_DATABASE = os.getenv("MYSQL_DATABASE")  
  
WINDOWS_SERVER = os.getenv("WIN_HOST")  
WINDOWS_USER = os.getenv("WIN_USER")  
WINDOWS_PASS = os.getenv("WIN_PASS")  
  
LINUX_SERVER = os.getenv("LINUX_HOST")  
LINUX_USER = os.getenv("LINUX_USER")  
LINUX_PASS = os.getenv("LINUX_PASS")
```

Le fichier **.env** contient toutes les informations nécessaires pour se connecter sur les différentes machines de l'infrastructure.

```
NTL-SysToolbox > ⚙️ .env
1  # =====
2  # CONFIGURATION MYSQL (Module Sauvegarde)
3  # =====
4  MYSQL_HOST=192.168.10.21
5  MYSQL_PORT=3306
6  MYSQL_USER=user1
7  MYSQL_PASSWORD=Password123!
8  MYSQL_DATABASE=wms_ntl
9
10
11 # =====
12 # CONFIGURATION DIAGNOSTIC
13 # =====
14 # Serveurs Active Directory (séparés par des virgules)
15 AD_SERVERS=192.168.10.10,192.168.10.11
16 AD_DOMAIN=epsi.local
17
18 # Serveurs DNS (séparés par des virgules)
19 DNS_SERVERS=192.168.10.10,192.168.10.11
```

La vérification DNS utilise la fonction **test_dns()**, qui vérifie la résolution du domaine en envoyant une requête DNS au serveur spécifié, puis affiche l'adresse IP renvoyée si la résolution réussit.

```
def test_dns(server_dns):
    """Teste la résolution DNS du domaine epsi.local"""
    print(f"[+] Vérification DNS du domaine epsi.local ({server_dns})...")
    try:
        resolver = dns.resolver.Resolver()
        resolver.nameservers = [server_dns]
        answer = resolver.resolve('epsi.local', 'A')
        print(f"  Résolution DNS OK : {answer[0]}\n")
    except Exception as e:
        print("  Erreur DNS :", e, "\n")
```

La fonction **test_ad()** contrôle quant à elle le bon fonctionnement du service Active Directory en vérifiant les ports principaux de l'Active Directory (LDAP, Kerberos, SMB).

```
def test_ad(ip):
    """Vérifie la disponibilité des services essentiels d'Active Directory"""
    print(f"[+] Vérification du service Active Directory sur epsi.local ({ip})...")

    # Liste des ports AD critiques
    ports = {
        389: "LDAP",
        636: "LDAP sécurisé (LDAPS)",
        88: "Kerberos",
        445: "NetLogon / SMB",
    }

    ad_ok = True

    for port, service in ports.items():
        try:
            sock = socket.create_connection((ip, port), timeout=3)
            print(f" [OK] {service} (port {port}) est accessible.")
            sock.close()
        except Exception as e:
            print(f" [ERREUR] {service} (port {port}) injoignable : {e}")
            ad_ok = False

    if ad_ok:
        print("Service Active Directory : tous les ports essentiels répondent.\n")
    else:
        print("Certains services AD sont inaccessibles. Vérifier la configuration ou le pare-feu.\n")
```

Pour vérifier le bon fonctionnement de la base MySQL, on teste la connexion à la base WMS via **mysql.connector**, avec les informations de connexion du **.env**.

```
def test_mysql():
    """Teste la connexion MySQL"""
    print(f"[+] Vérification de la base MySQL ({SERVER_MYSQL}) ...")
    try:
        conn = mysql.connector.connect(
            host=SERVER_MYSQL,
            user=MYSQL_USER,
            password=MYSQL_PASSWORD,
            database=MYSQL_DATABASE,
            connection_timeout=5
        )
        if conn.is_connected():
            print(" Connexion MySQL OK\n")
            conn.close()
    except Exception as e:
        print(" Erreur MySQL :", e, "\n")
```

Pour finir, les diagnostics pour obtenir des informations systèmes Windows et Ubuntu sont réalisés en initiant une connexion distante :

- Accès **Powershell** (commandes permettant de récupérer les informations) en se connectant via le protocole **WinRM** en transport **NTLM**, pour Windows

```
def diagnostic_windows(ip, username, password):
    """Collecte des infos système sur le Windows Server via WinRM"""
    print(f"=== Diagnostic de Windows Server ({ip}) ===")
    if not test_ping(ip):
        print("  Serveur injoignable.\n")
        return

    try:
        session = winrm.Session(f'http://{ip}:5985/wsman', auth=(username, password), transport='ntlm')
        ps_script = """
        $os = Get-CimInstance Win32_OperatingSystem
        $cpu = Get-Counter '\\Processor(_Total)\\% Processor Time' | Select -ExpandProperty CounterSamp
        $ram = [math]::Round(($os.TotalVisibleMemorySize - $os.FreePhysicalMemory) / $os.TotalVisibleM
        $disk = Get-WmiObject Win32_LogicalDisk -Filter "DeviceID='C:'" | Select-Object Size,FreeSpace
        $diskUsed = [math]::Round((1 - ($disk.FreeSpace / $disk.Size)) * 100, 2)
        $uptime = (Get-Date) - $os.LastBootUpTime
        $hours = [math]::Floor($uptime.TotalHours)
        $minutes = $uptime.Minutes
        Write-Output ("OS : " + $os.Caption)
        """

```

- Connexion **SSH** avec la bibliothèque **paramiko**, permettant d'exécuter des commandes système et de récupérer les informations, pour Linux.

```
def diagnostic_linux(ip, username, password):
    """Collecte des infos système sur le serveur Linux via SSH"""
    print(f"=== Diagnostic d'Ubuntu ({ip}) ===")
    if not test_ping(ip):
        print("  Machine injoignable.\n")
        return

    try:
        ssh = paramiko.SSHClient()
        ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
        ssh.connect(ip, username=username, password=password, timeout=5)

        commands = {
            "OS": "lsb_release -d | cut -f2",
            "Uptime": "uptime -p",
            "CPU": "top -bn1 | grep 'Cpu(s)' | awk '{print $2+$4}'",
            "RAM": "free | awk '/Mem/ {print $3/$2 * 100.0}'",
            "DISK": "df -h / | awk 'NR==2 {print $5}'"
        }

        for key, cmd in commands.items():
            stdin, stdout, stderr = ssh.exec_command(cmd)
            output = stdout.read().decode().strip()
            print(f"  {key}: {output}")

```

L'ensemble des fonctions sont ensuite appelées dans une seule fonction **main()**, qui lance lors de son exécution toutes les fonctionnalités du module 1. Les paramètres de ces fonctions correspondent aux **variables d'environnement** spécifiées précédemment.

```
def main():
    test_ping(SERVER_DNS)
    test_dns(SERVER_DNS)
    test_ad(SERVER_AD)
    test_mysql()
    diagnostic_windows(WINDOWS_SERVER, WINDOWS_USER, WINDOWS_PASS)
    diagnostic_linux(LINUX_SERVER, LINUX_USER, LINUX_PASS)
    diagnostic_linux(LINUX_CLIENT, LINUX_CLIENT_USER, LINUX_CLIENT_PASS)
    print("=== Diagnostic terminé ===")
```

4. Vérifications réalisées avec exemples de sorties

Plusieurs tests ont été réalisés sur l'infrastructure de test, dont les machines suivantes :

- Un serveur **Windows Server 2022** - 192.168.10.10
- Un serveur **Linux Ubuntu** - 192.168.10.21
- Un **client Ubuntu Desktop** - 192.168.10.50
- Un client **Windows 10** - 192.168.10.60
- Une base **MySQL WMS** hébergée sur 192.168.10.21

Dans l'exemple ci-dessous, nous testons d'abord la connexion à la machine agissant en tant que contrôleur de domaine (192.168.10.10), puis nous vérifions les services **DNS** et **Active Directory**, qui sont bien fonctionnels.

Par la suite, la **connexion à la base MySQL**, hébergée sur le serveur Linux (192.168.10.21) est vérifiée et s'avère fonctionnelle.

Enfin, des **diagnostics de systèmes**, à la fois **Windows Server** (192.168.10.10) et **Ubuntu** Server (192.168.10.21) et Client (192.168.10.50) sont réalisés. La sortie confirme le bon fonctionnement de la fonctionnalité, et renvoie les différentes informations des systèmes concernés (OS, uptime, RAM...).

Exemple de sortie

```
(venv_diag) user@user-VMware-Virtual-Platform:~$ python3 diagnostic.py
Chargement du .env ...
Windows server : 192.168.10.10
===== DIAGNOSTIC CENTRALISÉ =====

[+] Test de connexion vers 192.168.10.10 ...
    192.168.10.10 est joignable

[+] Vérification DNS du domaine epsi.local (192.168.10.10)...
    Résolution DNS OK : 192.168.10.10

[+] Vérification du service Active Directory sur epsi.local (192.168.10.10)...
    [OK] LDAP (port 389) est accessible.
    [OK] LDAP sécurisé (LDAPS) (port 636) est accessible.
    [OK] Kerberos (port 88) est accessible.
    [OK] NetLogon / SMB (port 445) est accessible.
    Service Active Directory : tous les ports essentiels répondent.

[+] Vérification de la base MySQL (192.168.10.21) ...
    Connexion MySQL OK

=== Diagnostic de Windows Server (192.168.10.10) ===
[+] Test de connexion vers 192.168.10.10 ...
    192.168.10.10 est joignable

OS : Microsoft Windows Server 2022 Datacenter Evaluation
Uptime: up 11 hours, 14 minutes
RAM : 73.65 %
Disk: 20.18 %

=== Diagnostic d'Ubuntu (192.168.10.21) ===
[+] Test de connexion vers 192.168.10.21 ...
    192.168.10.21 est joignable

    OS: Ubuntu 20.04.6 LTS
    Uptime: up 13 hours, 7 minutes
    CPU: 53
    RAM: 19.393
    DISK: 36%

=== Diagnostic d'Ubuntu (192.168.10.50) ===
[+] Test de connexion vers 192.168.10.50 ...
    192.168.10.50 est joignable

    OS: Ubuntu 24.04.3 LTS
    Uptime: up 2 minutes
    CPU: 42,1
    RAM: 32,3204
    DISK: 58%

=== Diagnostic terminé ===

[OK] Module Diagnostic terminé.

Appuyez sur Entrée pour revenir au menu... 
```


5. Complications rencontrées et limites

Quelques difficultés ont été rencontrées au cours du développement du module 1 :

- **Connexion WinRM** : la configuration du protocole Windows Remote Management nécessite des droits privilégiés pour se connecter à distance, ainsi que des configurations bien précises.

Le problème a été résolu principalement en respecifiant correctement le mot de passe défini dans le fichier .env, qui permet de se connecter au contrôleur de domaine.

```
=== Diagnostic serveur Windows (192.168.10.10) ===  
[+] Test de connexion vers 192.168.10.10 ...  
    192.168.10.10 est joignable  
  
    Erreur WinRM Windows : the specified credentials were rejected by the server
```

- **Temps d'exécution** : le diagnostic peut être plus lent sur un réseau comprenant plus de machines.
- **Permissions MySQL** : pour vérifier le service MySQL, il faut que l'utilisateur possède au minimum des privilèges PROCESS et LOCK TABLES. Dans le cas contraire, la vérification renvoie une erreur :

```
[+] Vérification de la base MySQL (192.168.10.21) ...  
    Erreur MySQL : 1045 (28000): Access denied for user 'user1'@'192.168.10.50'
```

- **Dépendance au fichier .env** : comme vu ci-dessus, en cas d'erreur de configuration (mot de passe, IP incorrecte) le diagnostic renvoie une erreur.

V. Module 2 : Sauvegarde WMS

1. Objectifs et fonctionnalités

Le module de **sauvegarde WMS** a pour objectif d'assurer la **sauvegarde, l'intégrité et la traçabilité** des données de la base de données de gestion d'entrepôt (WMS). Il permet de sécuriser les données métiers et de limiter les pertes d'informations en cas d'incident.

Le module assure deux fonctionnalités :

- Sauvegarde de l'ensemble de la base de données au format **.sql**
- Export d'une table spécifiée au format **.csv**

2. Processus de sauvegarde

Le script fait appel à plusieurs bibliothèques Python :

- **mysql.connector** : permet de se connecter à la base MySQL et d'exécuter des requêtes
- **subprocess** : permet d'exécuter la commande système mysqldump
- **csv** : utile pour la génération au format CSV
- **dotenv** : qui comme vu précédemment, charge les variables d'environnement définies dans le fichier .env.

Le code est structuré en deux fonctions distinctes, correspondant chacune à une fonctionnalité du module.

La fonction **sauvegarde_sql()** crée un fichier **.sql** contenant une copie complète de la base. Elle définit le nom du fichier de sauvegarde à générer, en prenant en compte l'**horodatage** (date + heure) pour éviter d'écraser de potentielles anciennes sauvegardes.

Elle prépare ensuite l'exécution de la commande **mysqldump**, qui permet d'exporter la base de données (structure et contenu), en spécifiant plusieurs paramètres tels que l'adresse IP de l'hôte et les identifiants pour se connecter à la base.

La commande mysqldump est ensuite exécutée via **subprocess.run()**, et sa sortie (le contenu SQL) est **redirigée** dans le **fichier de sauvegarde**.

```
def sauvegarde_sql():
    """Effectue une sauvegarde complète de la base MySQL au format .sql"""
    try:
        if not os.path.exists(BACKUP_DIR):
            os.makedirs(BACKUP_DIR)

        # Nom du fichier avec horodatage
        backup_file = os.path.join(
            BACKUP_DIR, f"{DB_NAME}_backup_{datetime.now().strftime('%Y%m%d_%H%M%S')}.sql"
        )

        print(f"[+] Sauvegarde de la base {DB_NAME} en cours...")
        cmd = [
            "mysqldump",
            f"-h{DB_HOST}",
            f"-u{DB_USER}",
            f"-p{DB_PASSWORD}",
            DB_NAME,
        ]

        with open(backup_file, "w", encoding="utf-8") as f:
            subprocess.run(cmd, stdout=f, check=True)

        print(f"[OK] Sauvegarde SQL enregistrée dans : {backup_file}\n")

    except subprocess.CalledProcessError as e:
        print(f"[ERREUR] Échec de la sauvegarde SQL : {e}")
    except Exception as e:
        print(f"[ERREUR] Problème inattendu : {e}")
```

La deuxième fonction, **export_table_csv()**, permet d'exporter une table que l'on saisis manuellement lors de l'exécution du module. Il est donc nécessaire de connaître les tables qui composent la base de donnée, étant donné que l'outil est destiné aux employés habilités connaissant la structure de la base.

Comme la première fonction, le nom du fichier est configuré pour contenir le nom de la table et un **horodatage** (date et heure) pour une meilleure traçabilité.

Par la suite, la fonction initie une connexion avec la base avec les informations stockées dans le fichier **.env**, puis récupère la table souhaitée à l'aide d'une requête SQL exécutée grâce à un curseur (**SELECT * FROM table_name**).

Le contenu de la table est ensuite collé dans le fichier CSV créé.

```
def export_table_csv(table_name=None):
    """Exporte une table MySQL au format CSV"""
    try:
        if not os.path.exists(BACKUP_DIR):
            os.makedirs(BACKUP_DIR)

        # Si la table n'est pas fournie, on demande à l'utilisateur
        if not table_name:
            table_name = input("Entrez le nom de la table à exporter : ").strip()

        if not table_name:
            print("[ERREUR] Nom de table vide. Export annulé.\n")
            return

        csv_file = os.path.join(
            BACKUP_DIR, f"{table_name}_export_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"
        )

        print(f"[+] Export de la table '{table_name}' en CSV...")

        # Connexion à MySQL
        conn = mysql.connector.connect(
            host=DB_HOST,
            user=DB_USER,
            password=DB_PASSWORD,
            database=DB_NAME
        )

        cursor = conn.cursor()
        cursor.execute(f"SELECT * FROM {table_name}")
        rows = cursor.fetchall()
        headers = [desc[0] for desc in cursor.description]

        # Écriture dans le fichier CSV
        with open(csv_file, "w", newline="", encoding="utf-8") as f:
            writer = csv.writer(f)
            writer.writerow(headers)
```

3. Vérification et exemples de sorties

Au lancement du module, la sauvegarde de la base s'effectue automatiquement. Pour l'export de table, il est ensuite nécessaire de spécifier le nom de la table à exporter (voir VII-2. pour la liste des tables disponibles).

```
[INFO] Exécution du module Sauvegarde WMS...

[+] Sauvegarde de la base wms_ntl en cours...
mysqldump: [Warning] Using a password on the command line interface can be insecure.
[OK] Sauvegarde SQL enregistrée dans : ./backups/wms_ntl_backup_20260106_121214.sql

Entrez le nom de la table à exporter : products
[+] Export de la table 'products' en CSV...
[OK] Export CSV enregistré dans : ./backups/products_export_20260106_121255.csv

[OK] Module Sauvegarde terminé.

Appuyez sur Entrée pour revenir au menu...
```

On retrouve les fichiers générés dans le dossier dédié **backups/**, où on retrouve la sauvegarde de la base MySQL ainsi que l'export CSV de la table choisie '**products**'.

```
user@user-VMware-Virtual-Platform:~/NTL-SysToolbox/backups$ cat wms_ntl_backup_20260106_121214.sql
-- MySQL dump 10.13 Distrib 8.0.44, for Linux (x86_64)
--
-- Host: 192.168.10.21 Database: wms_ntl
--
-- Server version      8.0.42-0ubuntu0.20.04.1

/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!50503 SET NAMES utf8mb4 */;
/*!40103 SET @OLD_TIME_ZONE=@@TIME_ZONE */;
/*!40103 SET TIME_ZONE='+00:00' */;
/*!40014 SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0 */;
/*!40014 SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0 */;
/*!40101 SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='NO_AUTO_VALUE_ON_ZERO' */;
/*!40111 SET @OLD_SQL_NOTES=@@SQL_NOTES, SQL_NOTES=0 */;

--
-- Table structure for table `carriers`
--

DROP TABLE IF EXISTS `carriers`;
/*!40101 SET @saved_cs_client = @@character_set_client */;
/*!50503 SET character_set_client = utf8mb4 */;
CREATE TABLE `carriers` (
  `carrier_id` int NOT NULL AUTO_INCREMENT,
  `carrier_code` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `carrier_name` varchar(100) COLLATE utf8mb4_unicode_ci NOT NULL,
  `contact_email` varchar(100) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `contact_phone` varchar(20) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `edi_enabled` tinyint(1) DEFAULT '0',
  `api_endpoint` varchar(255) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `is_active` tinyint(1) DEFAULT '1',
  `created_at` timestamp NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`carrier_id`),
  UNIQUE KEY `carrier_code` (`carrier_code`)
) ENGINE=InnoDB;

user@user-VMware-Virtual-Platform:~/NTL-SysToolbox/backups$ cat products_export_20260106_121255.csv
product_id,sku,product_name,category,weight_kg,dimensions,barcode,is_fragile,requires_cold_chain,created_at,updated_at
1,ELEC-001,Ordinateur portable Dell XPS 15,Electronics,2.500,35x24x2cm,3800123456789,1,0,2025-12-16 11:25:00,2025-12-16 11:25:00
2,ELEC-002,Smartphone Samsung Galaxy S24,Electronics,0.200,15x7x1cm,3800123456790,1,0,2025-12-16 11:25:00,2025-12-16 11:25:00
3,FURN-001,Chaise de bureau ergonomique,Furniture,12.000,60x60x120cm,3800123456791,0,0,2025-12-16 11:25:00,2025-12-16 11:25:00
4,FURN-002,Bureau réglable en hauteur,Furniture,45.000,160x80x75cm,3800123456792,0,0,2025-12-16 11:25:00,2025-12-16 11:25:00
5,BOOK-001,"Livre ""Gestion de la Supply Chain""",Books,0.800,24x16x3cm,9782100819546,0,0,2025-12-16 11:25:00,2025-12-16 11:25:00
6,FOOD-001,Chocolats artisanaux premium,Food,0.500,20x15x5cm,3800123456793,1,1,2025-12-16 11:25:00,2025-12-16 11:25:00
7,CLOTH-001,Veste imperméable homme,Clothing,0.600,50x40x10cm,3800123456794,0,0,2025-12-16 11:25:00,2025-12-16 11:25:00
8,TOYS-001,Jeu de construction LEGO Technic,Toys,2.200,38x26x7cm,5702016617481,0,0,2025-12-16 11:25:00,2025-12-16 11:25:00
9,ELEC-003,"Tablette iPad Pro 12.9""",Electronics,0.700,28x21x1cm,3800123456795,1,0,2025-12-16 11:25:00,2025-12-16 11:25:00
10,SPORT-001,Vélo électrique urbain,Sports,22.000,180x110x70cm,3800123456796,0,0,2025-12-16 11:25:00,2025-12-16 11:25:00
```

4. Sécurité et traçabilité

L'utilisation du fichier **.env** pour stocker les informations de connexion de façon **séparée du code source** permet d'assurer la **sécurité d'accès** à la base MySQL.

En effet, les mots de passe n'apparaissent jamais dans le script ou dans l'historique du terminal, ce qui réduit les risques.

De plus, la **traçabilité** est garantie par plusieurs dispositifs mis en place, tels que l'**horodatage des noms de fichiers**, les indications visibles dans la sortie du script et permettant de suivre chaque étape, ainsi que le **stockage centralisé des fichiers de sauvegarde ou d'export** dans le dossier dédié **out/backups/**.

```
user@user-VMware-Virtual-Platform:~/NTL-SysToolbox$ cd backups
user@user-VMware-Virtual-Platform:~/NTL-SysToolbox/backups$ ls
None_backup_20251219_160710.sql      products_export_20251219_164626.csv  wms_ntl_backup_20251219_163330.sql
products_export_20251219_162050.csv  products_export_20251219_165604.csv  wms_ntl_backup_20251219_164553.sql
products_export_20251219_162933.csv  products_export_20260106_121255.csv  wms_ntl_backup_20251219_164621.sql
products_export_20251219_163331.csv  wms_ntl_backup_20251219_162050.sql   wms_ntl_backup_20251219_165544.sql
products_export_20251219_164558.csv  wms_ntl_backup_20251219_162933.sql   wms_ntl_backup_20260106_121214.sql
```

5. Complications rencontrées et limites

Plusieurs difficultés se sont présentées en implémentant ce module :

- **Permissions MySQL** : l'utilisateur MySQL devait posséder le privilège **PROCESS** pour utiliser **mysqldump**.

Le problème a été résolu en modifiant les droits détenus par l'utilisateur de MySQL, sur le serveur de base de données

```
[+] Sauvegarde de la base wms_ntl en cours...
mysqldump: [Warning] Using a password on the command line interface can be insecure.
mysqldump: Error: 'Access denied; you need (at least one of) the PROCESS privilege(s) for this operation'
dump tablespaces
```

- **Sécurité de la commande mysqldump** : l'exécution de cette commande avec mot de passe en ligne de commande affiche un avertissement de sécurité (risque d'exposition dans l'historique)

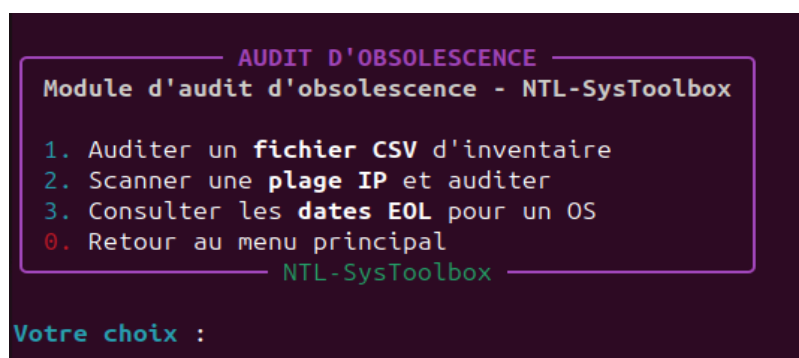
```
[+] Sauvegarde de la base wms_ntl en cours...
mysqldump: [Warning] Using a password on the command line interface can be insecure.
```

VI. Module 3 : Audit d'obsolescence

Le module d'audit d'obsolescence a pour objectif d'aider la DSI à identifier les équipements dont le système d'exploitation arrive en fin de vie ou n'est plus supporté.

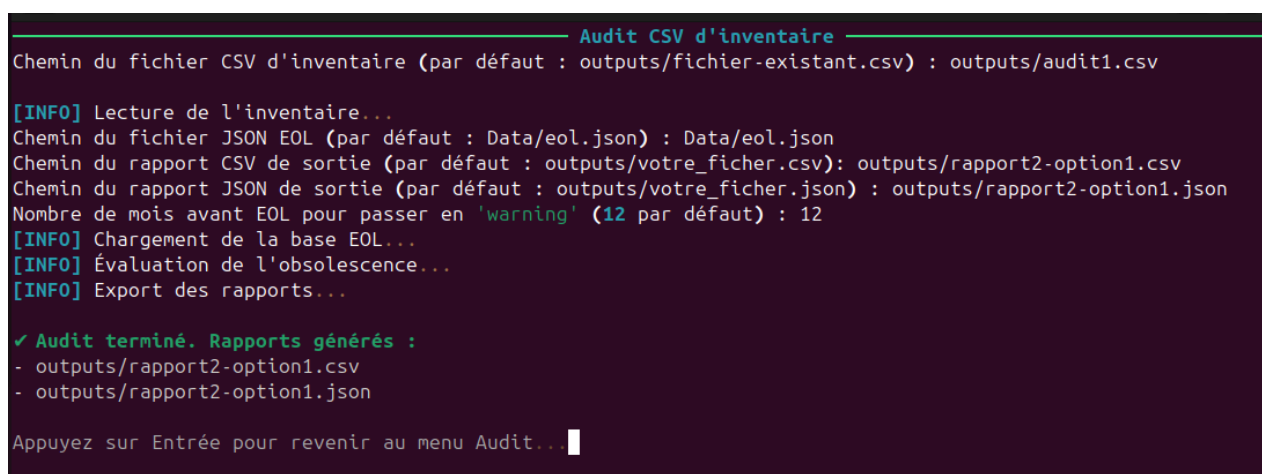
Il repose sur l'analyse d'un **inventaire réseau** et la comparaison des versions détectées avec une base de données **EOL** (End Of Life).

Trois fonctionnalités sont disponibles :



1) Auditer un fichier CSV d'inventaire

L'utilisateur sélectionne un fichier CSV contenant les machines à auditer (par exemple, **outputs/fichier-existant.csv**).



La fonction `run_csv_audit_flow()` permet de lire ce fichier, puis charge la base des dates EOL (**Data/eol.json**) et évalue chaque système.


```
def run_csv_audit_flow() -> None:
    """Audit à partir d'un inventaire CSV."""
    #inventory_path = console.input("Chemin du fichier CSV d'inventaire (par défaut : outputs/fichier-existant.csv) : ").strip()
    inventory_path = ask_existing_file(
        "Chemin du fichier CSV d'inventaire (par défaut : outputs/fichier-existant.csv) : ",
        "outputs/fichier-existant.csv")

    console.print("\n[bold cyan][INFO][/bold cyan] Lecture de l'inventaire...")
    components = parse_inventory_csv(inventory_path)

    #eol_path = console.input("Chemin du fichier JSON EOL (par défaut : Data/eol.json) : ").strip()
    eol_path = ask_existing_file(
        "Chemin du fichier JSON EOL (par défaut : Data/eol.json) : ",
        "Data/eol.json")

    output_csv = console.input("Chemin du rapport CSV de sortie (par défaut : outputs/votre_fichier.csv): ").strip()
    output_json = console.input("Chemin du rapport JSON de sortie (par défaut : outputs/votre_fichier.json) : ").strip()
    warning_months_str = console.input(
        "Nombre de mois avant EOL pour passer en 'warning' (12 par défaut) : "
    ).strip()
    warning_months = int(warning_months_str) if warning_months_str else 12

    console.print("[bold cyan][INFO][/bold cyan] Chargement de la base EOL...")
    eol_db = load_eol_database(eol_path)

    console.print("[bold cyan][INFO][/bold cyan] Évaluation de l'obsolescence...")
    evaluated = evaluate_components(components, eol_db, warning_months=warning_months)
```

2) Scanner une plage IP et auditer

Le script du module 3 permet de lancer un scan Nmap sur une plage d'adresses (par exemple : **192.168.10.0/24**) pour détecter les hôtes actifs.

Les résultats (IP, hostname, OS, version) sont stockés dans une liste de dictionnaires Python. Ils sont ensuite comparés à la base **Data/eol.json** pour évaluer leur niveau de support.

```

Scan réseau + audit
Plage réseau à scanner (ex: 192.168.56.0/24) : 192.168.10.0/24
[INFO] Scan réseau sur 192.168.10.0/24 ...
[INFO] 6 machine(s) détectée(s).
Chemin du fichier JSON EOL (par défaut : Data/eol.json) : Data/eol.json
Chemin du rapport CSV de sortie (par défaut : outputs/votre_fichier.csv) : outputs/audit2.csv
Chemin du rapport JSON de sortie (par défaut : outputs/votre_fichier.json) : outputs/audit2.json
Nombre de mois avant EOL pour passer en 'warning' (12 par défaut) : 12
[INFO] Chargement de la base EOL...
[INFO] Évaluation de l'obsolescence...
[INFO] Export des rapports...

✓ Audit terminé. Rapports générés :
- outputs/audit2.csv
- outputs/audit2.json

Appuyez sur Entrée pour revenir au menu Audit...
```

Exemple de rapport généré :

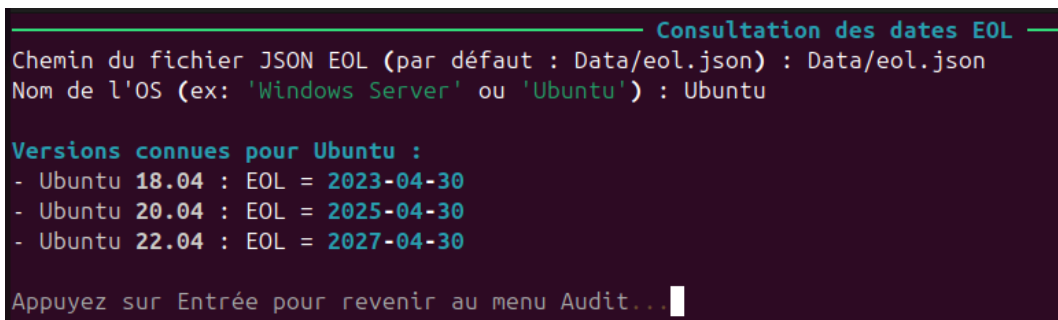
```
hostname,ip,os,version,eol_date,status
192.168.10.1,192.168.10.1,Microsoft Windows 11
21H2,Unknown,,unknown
WIN-4IKJKPO3FBC.epsi.local,192.168.10.10,Microsoft Windows
Server 2022,Unknown,2031-10-14,supported
192.168.10.2,192.168.10.2,VMware Player virtual NAT
device,Unknown,,unknown
192.168.10.21,192.168.10.21,Linux 4.15 - 5.19,Unknown,,unknown
192.168.10.254,192.168.10.254,Unknown,Unknown,,unknown
192.168.10.50,192.168.10.50,Linux 4.15 - 5.19,Unknown,,unknown
```


3) Consulter dates EOL pour un OS donné

Permet d'interroger directement le fichier **Data/eol.json** pour afficher la liste des versions et leurs dates de fin de vie.

```
def list_os_versions(os_name: str, eol_db: Dict[str, Dict[str, str]]) -> None:
    """
    Affiche toutes les versions d'un OS et leurs dates EOL
    à partir de la base EOL.
    """
    versions = eol_db.get(os_name)
    if not versions:
        console.print(f"[bold yellow][WARN][/] Aucun OS nommé '{os_name}' dans la base EOL.")
        return

    console.print(f"[bold cyan]Versions connues pour {os_name} :[/bold cyan]")
    for version, eol in versions.items():
        console.print(f"- [white]{os_name} {version}[/white] : EOL = [bold]{eol}[/bold]")
```



```
Consultation des dates EOL
Chemin du fichier JSON EOL (par défaut : Data/eol.json) : Data/eol.json
Nom de l'OS (ex: 'Windows Server' ou 'Ubuntu') : Ubuntu

Versions connues pour Ubuntu :
- Ubuntu 18.04 : EOL = 2023-04-30
- Ubuntu 20.04 : EOL = 2025-04-30
- Ubuntu 22.04 : EOL = 2027-04-30

Appuyez sur Entrée pour revenir au menu Audit...
```

1. Méthodologie d'inventaire

L'inventaire est réalisé automatiquement à partir d'un scan réseau de la plage IP fournie, en s'appuyant sur Nmap pour détecter les hôtes actifs et leurs systèmes d'exploitation.

Les résultats sont ensuite stockés dans un dictionnaire Python et exportés sous forme de rapports CSV et JSON.

La fonction **scan_network_simple()** interroge chaque hôte de la plage IP indiquée, détecte son système d'exploitation à l'aide de **Nmap**, puis enregistre les informations collectées dans une structure Python avant l'export des rapports.

```
def scan_network_simple(cidr: str) -> List[Dict[str, str]]:
    """
    Scanne une plage réseau (ex: '192.168.56.0/24') avec Nmap
    et renvoie une liste de machines au format:
    hostname, ip, os, version.
    """
    scanner = nmap.PortScanner()
    console.print(f"[bold cyan][INFO][/bold cyan] Scan réseau sur [white]{cidr}[/white] ...")
    scanner.scan(hosts=cidr, arguments='-O -T4') # -O = détection OS

    components: List[Dict[str, str]] = []

    for host in scanner.all_hosts():
        ip = host
        hostname = scanner[host].hostname() or host

        detected_os = "Unknown"
        detected_version = "Unknown"

        if "osmatch" in scanner[host] and scanner[host]["osmatch"]:
            osmatch = scanner[host]["osmatch"][0]
            os_name = osmatch.get("name", "")
            detected_os = os_name or "Unknown"

        components.append(
            {
                "hostname": hostname,
                "ip": ip,
                "os": detected_os,
                "version": detected_version,
            }
        )

    return components
```

2. Détection OS

La détection des systèmes d'exploitation se fait avec Nmap, qui identifie l'OS des machines (**option -O**) en comparant leurs réponses réseau à une base de modèles connus.

Cela permet d'identifier automatiquement des systèmes comme Windows Server ou Ubuntu, même en l'absence de connexion SSH ou WinRM.

```
scanner.scan(hosts=cidr, arguments='-O -T4')
```

```
if "osmatch" in scanner[host] and scanner[host]["osmatch"]:
    osmatch = scanner[host]["osmatch"][0]
    os_name = osmatch.get("name", "")
    detected_os = os_name or "Unknown"
```

3. Source EOL (éditeur)

Les dates de fin de support (EOL) sont stockées dans un fichier JSON **Data/eol.json**, maintenu par l'administrateur ou par la DSI à partir des sources officielles des éditeurs (Microsoft, Canonical, Linux notamment).

Lors de l'exécution de l'audit d'obsolescence, chaque système détecté est comparé à cette base pour déterminer son statut de support.

NTL-SysToolbox > Data > {} eol.json > ...

```
1 {
2   "Windows Server": {
3     "2012": "2023-10-10",
4     "2016": "2027-01-12",
5     "2019": "2029-01-09",
6     "2022": "2031-10-14"
7   },
8   "Ubuntu": {
9     "18.04": "2023-04-30",
10    "20.04": "2025-04-30",
11    "22.04": "2027-04-30"
12  }
13 }
```

```
Consultation des dates EOL
Chemin du fichier JSON EOL (par défaut : Data/eol.json) : Data/eol.json
Nom de l'OS (ex: 'Windows Server' ou 'Ubuntu') : Windows Server

Versions connues pour Windows Server :
- Windows Server 2012 : EOL = 2023-10-10
- Windows Server 2016 : EOL = 2027-01-12
- Windows Server 2019 : EOL = 2029-01-09
- Windows Server 2022 : EOL = 2031-10-14

Appuyez sur Entrée pour revenir au menu Audit...
```

4. Classification des risques

La classification des risques consiste à regrouper les systèmes en catégories (supporté / bientôt en fin de support / déjà en fin de support) afin de prioriser les actions à mener :

- Si **eol_date < aujourd'hui** : **déjà en fin de support** (obsolète, risque élevé).
- Si **0 ≤ (eol_date - today) ≤ seuil_en_mois** : **bientôt en fin de support** (warning, risque moyen / à surveiller).

- Si $(eol_date - today) > \text{seuil_en_mois}$: **supporté** (risque faible).
- Si **aucune date EOL** n'est trouvée pour cet OS : **inconnu / non référencé** (risque à évaluer manuellement).

Cette classification est automatiquement appliquée par la fonction **classify_component()** du script.

5. Exemple de rapport

	A	B	C	D	E	F	G
1	hostname,ip,os,version,eol_date,status						
2	192.168.19.1	192.168.19.1	Microsoft Windows 10 1607 - 11 23H2	Unknown	unknown		
3	192.168.19.128	192.168.19.128	Linux 4.15 - 5.19	Unknown	unknown		
4	192.168.19.129	192.168.19.129	Microsoft Windows Server 2022	Unknown	2031-10-14	supported	
5	192.168.19.254	192.168.19.254	Unknown	Unknown	unknown		

```

{
  "generated_at": "2026-01-08T11:41:30.429202Z",
  "components": [
    {
      "hostname": "192.168.19.1",
      "ip": "192.168.19.1",
      "os": "Microsoft Windows 10 1607 - 11 23H2",
      "version": "Unknown",
      "eol_date": null,
      "status": "unknown"
    },
    {
      "hostname": "192.168.19.129",
      "ip": "192.168.19.129",
      "os": "Microsoft Windows Server 2022",
      "version": "Unknown",
      "eol_date": "2031-10-14",
      "status": "supported"
    },
    {
      "hostname": "192.168.19.254",
      "ip": "192.168.19.254",
      "os": "Unknown",
      "version": "Unknown",
      "eol_date": null,
      "status": "unknown"
    }
  ]
}

```



```

Audit CSV d'inventaire
Chemin du fichier CSV d'inventaire (par défaut : outputs/fichier-existant.csv) : abcd
✖ Fichier introuvable : abcd
Appuyez sur Entrée pour réessayer...

Chemin du fichier CSV d'inventaire (par défaut : outputs/fichier-existant.csv) : efgh
✖ Fichier introuvable : efgh
Appuyez sur Entrée pour réessayer...

Chemin du fichier CSV d'inventaire (par défaut : outputs/fichier-existant.csv) : outputs/inventaire1.csv

[INFO] Lecture de l'inventaire...
Chemin du fichier JSON EOL (par défaut : Data/eol.json) : Data/eol.json

```

- **Problème d’affichage sous Windows** : la bibliothèque Rich utilisée sous Windows causait un problème d’affichage au niveau du menu du module 3, qui restait affiché en fond, même lorsqu’une option était sélectionnée.

```
Scan réseau + audit  
Plage réseau à scanner (ex: 192.168.10.0/24) : 192.168.10.0/24  
[INFO] Scan réseau sur 192.168.10.0/24 ...  
  
- - - - -  
AUDIT D'OBSOLESCENCE  
Module d'audit d'obsolescence - NTL-SysToolbox  
  
1. Auditer un fichier CSV d'inventaire  
2. Scanner une plage IP et auditer  
3. Consulter les dates EOL pour un OS  
0. Retour au menu principal  
NTL-SysToolbox  
  
Votre choix : 2
```

Cela a été corrigé en ajoutant une fonction **clear-screen()**, permettant de dissocier la commande utilisée sur Windows (commande native **cls**) de celle de Linux (**console.clear()**) pour réinitialiser correctement le terminal.

```
# Gestion de l'affichage de la console du module
import os
def clear_screen():
    if os.name == "nt":    # Windows
        os.system("cls")
    else:
        console.clear()
```

VII. Mise en place de l'infrastructure de test

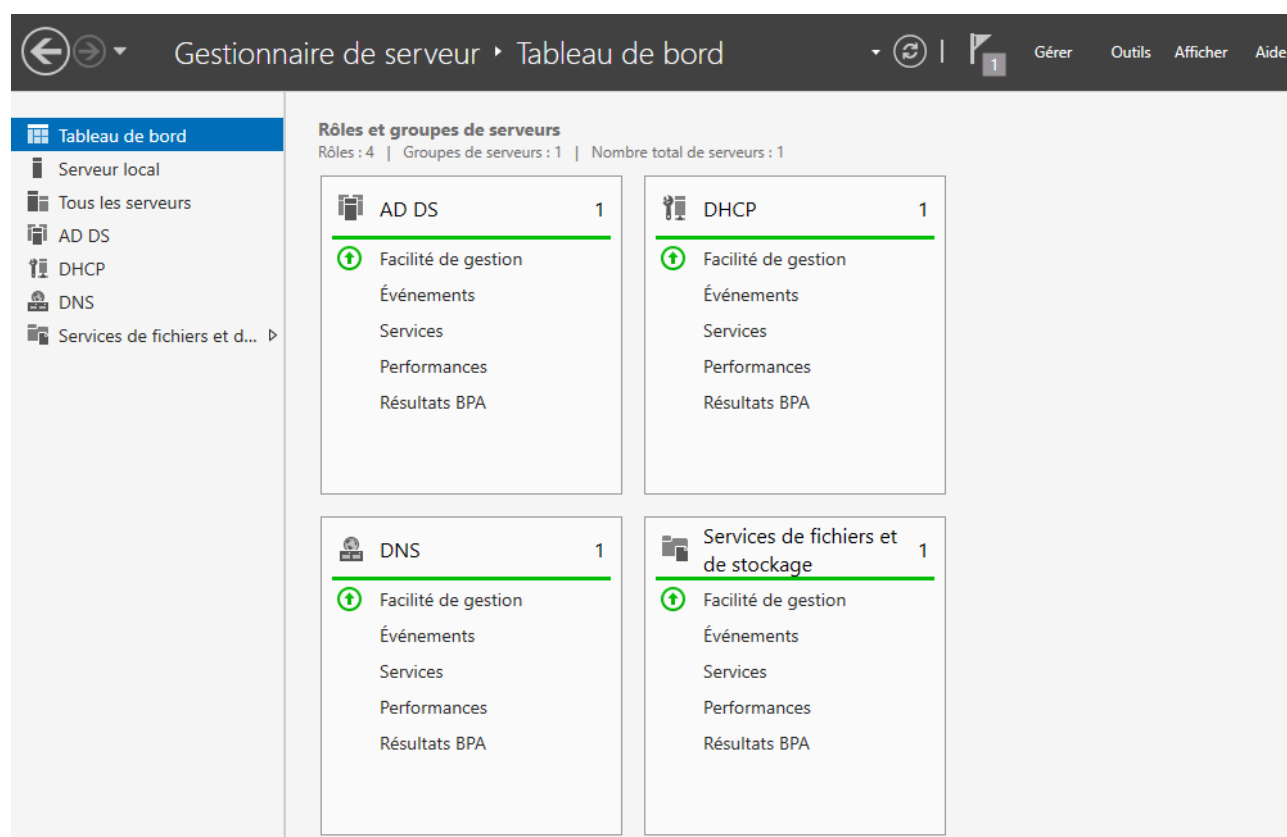
L'infrastructure de test a été virtualisée sous VMWare. Elle reproduit de manière simplifiée l'infrastructure de NTL, en conservant les éléments qui contribuent au fonctionnement ainsi qu'aux tests des trois fonctionnalités de l'outil.

Comme présenté précédemment, celle-ci est composée d'un contrôleur de domaine, assurant les services AD et DNS, ainsi que de machines clientes Windows et Linux.

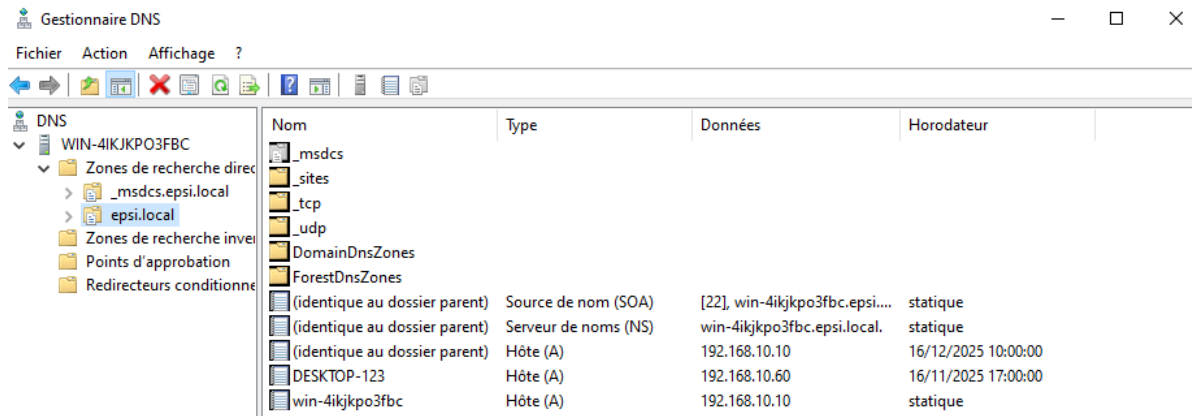
1. Mise en place Active Directory et DNS

Un contrôleur de domaine principal a été déployé sous Windows Server : il centralise l'authentification et la résolution de noms pour les machines du domaine.

L'installation des rôles Active Directory (AD DS) et DNS a été réalisée via le *Gestionnaire de serveur*, qui permet de créer le domaine interne.



Après avoir installé les services, le serveur a été promu en contrôleur de domaine, puis une zone DNS directe a été configurée, pour permettre la résolution de nom entre les différentes machines du domaine.



Pour tester le bon fonctionnement du DNS, nous avons également pu utiliser la commande **nslookup**, qui confirme sa disponibilité.

2. Mise en place serveur Linux de base de données

Le serveur Linux Ubuntu a été mis en place afin d'héberger la base WMS. La base a été transférée de la machine locale vers la machine virtuelle grâce à la commande scp, permettant un transfert distant sécurisé, puis importée dans mysql.

```
PS C:\Users\laura> scp "C:\Users\laura\Downloads\base.sql" user@192.168.10.21:/tmp/
The authenticity of host '192.168.10.21 (192.168.10.21)' can't be established.
ED25519 key fingerprint is SHA256:53LAWB1GMSuDjR4LtejbZKGjjh20+McjlezIiomBfJg.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
Warning: Permanently added '192.168.10.21' (ED25519) to the list of known hosts.
user@192.168.10.21's password:
```

```
user@linuxserver:/tmp$ sudo mysql < /tmp/base.sql
status
Base de données WMS créée avec succès!
```

```
mysql> SHOW TABLES;
```

```
+-----+
| Tables_in_wms_ntl |
+-----+
| carriers           |
| customers          |
| inventory          |
| movements          |
| order_lines        |
| orders             |
| products           |
| system_logs        |
| users              |
| v_active_orders    |
| v_daily_activity   |
| v_inventory_summary|
| warehouses         |
```


Un utilisateur a également été créé pour permettre au module de sauvegarde ou de diagnostic de se connecter à la base de données, sans utiliser directement le compte administrateur, et éviter ainsi des risques de sécurité.

Pour assurer le fonctionnement des tests, l'utilisateur peut se connecter depuis n'importe quelle machine du réseau (spécifié avec @% lors de sa création) et possède des droits avancés.

```
mysql> CREATE USER 'user1'@'%' IDENTIFIED BY 'Password123!';
Query OK, 0 rows affected (0,01 sec)

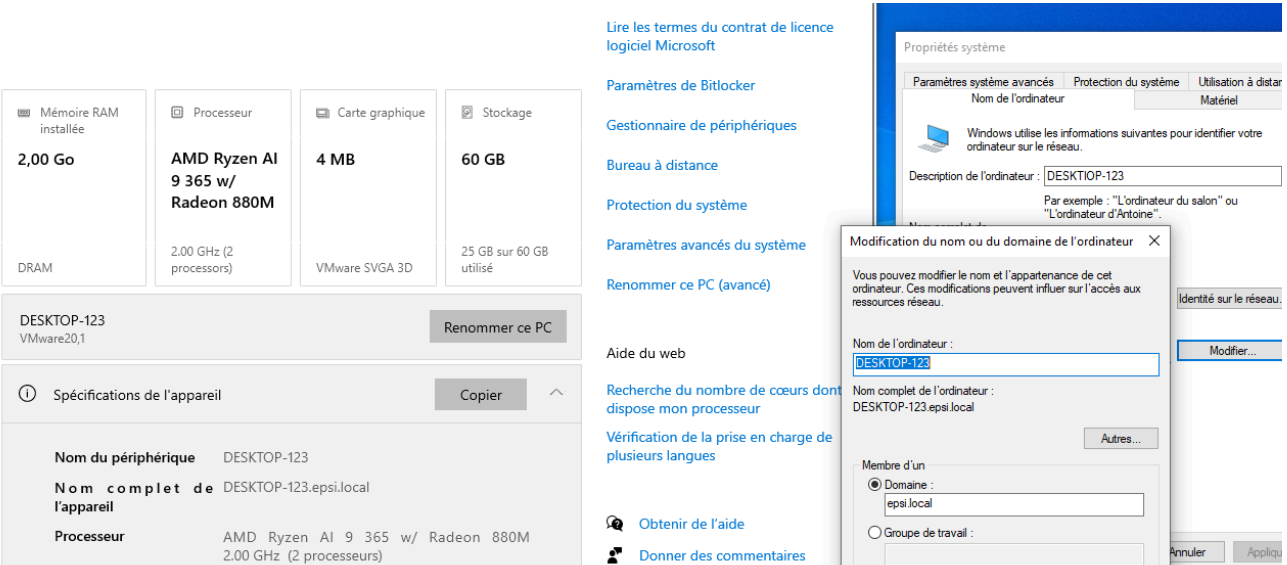
mysql> GRANT ALL PRIVILEGES ON wms_ntl.* TO 'user1'@'%' ;
Query OK, 0 rows affected (0,00 sec)
```

3. Machines clientes

Deux machines clientes ont également été déployées pour tester les fonctionnalités du module 3, qui réalise un audit d'obsolescence des composants du réseau.

a. Windows

La machine Windows a été installée sous Windows 10. Après avoir réalisé les configurations d'installation, la machine a été intégrée au domaine Active Directory pour tester la communication avec le contrôleur de domaine, et exécuter le script depuis un environnement du domaine.



The screenshot displays the Windows 'System' settings window. The left sidebar shows 'Spécifications de l'appareil' (Device specifications) with details on RAM (2.00 Go), Processor (AMD Ryzen AI 9 365 w/ Radeon 880M), Graphics (4 MB), and Storage (60 GB). The right pane shows 'Propriétés système' (System properties) with the 'Nom de l'ordinateur' (Computer name) set to 'DESKTOP-123' and the 'Nom complet de l'ordinateur' (Full computer name) set to 'DESKTOP-123.epsi.local'. A 'Modification du nom ou du domaine de l'ordinateur' (Change computer name or domain) dialog box is open, showing the current name 'DESKTOP-123' and domain 'epsi.local'.

Les tests de connectivité (commande ping) ont permis de vérifier la communication de la machine avec le serveur Active Directory et le DNS.

```
C:\Users\laurap>ping epsi.local

Envoi d'une requête 'ping' sur epsi.local [192.168.10.10] avec 32 octets de données :
Réponse de 192.168.10.10 : octets=32 temps=1 ms TTL=128
Réponse de 192.168.10.10 : octets=32 temps<1ms TTL=128
```

b. Linux - Ubuntu Desktop (24.0)

La machine cliente Linux a pour sa part été installée sous Ubuntu Desktop. Après l'installation, les paramètres réseau ont été configurés afin que les requêtes DNS de la machine soient résolues via le serveur Active Directory.

```
user@user-VMware-Virtual-Platform:~$ sudo systemctl stop systemd-resolved
[sudo] Mot de passe de user :
user@user-VMware-Virtual-Platform:~$ sudo systemctl disable systemd-resolved
Removed "/etc/systemd/system/sysinit.target.wants/systemd-resolved.service".
Removed "/etc/systemd/system/dbus-org.freedesktop.resolve1.service".
user@user-VMware-Virtual-Platform:~$ sudo rm /etc/resolv.conf
user@user-VMware-Virtual-Platform:~$ echo "nameserver 192.168.10.10" | sudo tee /etc/resolv.conf
> ^C
user@user-VMware-Virtual-Platform:~$ echo "nameserver 192.168.10.10" | sudo tee /etc/resolv.conf
```

```
user@user-VMware-Virtual-Platform:~$ nslookup epsi.local
Server:          192.168.10.10
Address:         192.168.10.10#53

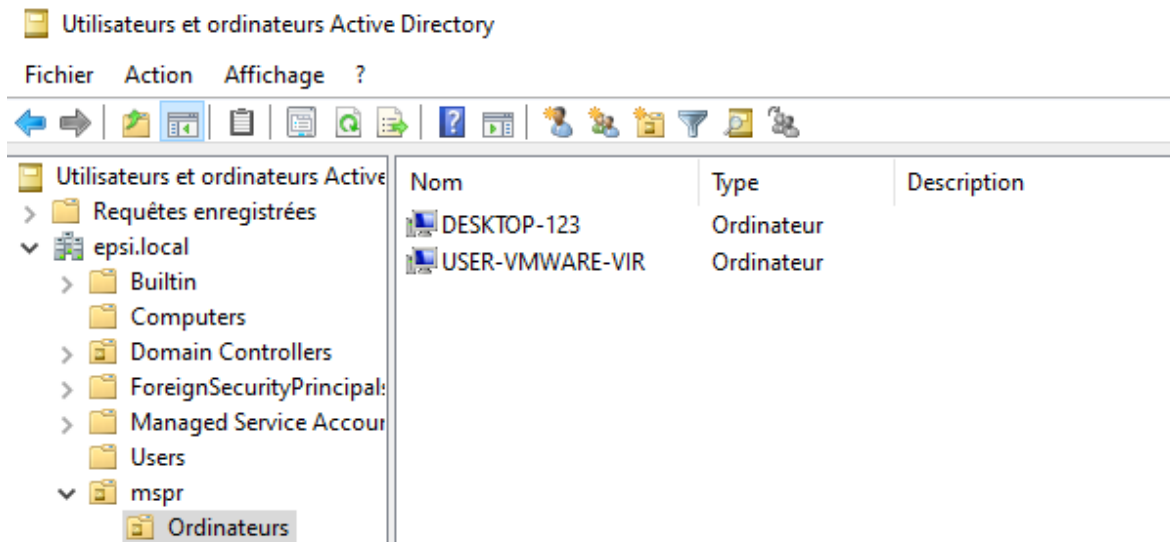
Name:   epsi.local
Address: 192.168.10.10
```

Une fois la connexion au domaine fonctionnelle, la machine a été ajoutée dans l'Active Directory. Dans la commande ci-dessous, le compte Administrateur du domaine crée ici automatiquement l'objet ordinateur dans l'AD, dans le dossier dédié spécifié.

```
user@user-VMware-Virtual-Platform:~$ sudo realm join epsi.local -U Administrateur --computer-ou="OU=Ordinateurs,OU=mspr,DC=epsi,DC=local"
Password for Administrateur:
* Installing necessary packages: sssd-tools
```

c. Vérification de l'ajout des machines au domaine

Les deux machines clientes Windows et Linux font maintenant partie des ordinateurs inclus dans le domaine Active Directory.



VIII. Axes d'amélioration futures

Bien que l'outil **NTL-SysToolbox** soit fonctionnel, plusieurs axes d'améliorations peuvent être explorés pour enrichir ses fonctionnalités et faciliter encore davantage son intégration dans l'environnement informatique de NTL.

Interface graphique

Une interface graphique simplifiée, par exemple sous la forme d'un **tableau de bord web**, pourrait être développée, afin de permettre aux techniciens et responsables IT d'exécuter les modules sans nécessité de passer par la ligne de commande.

Cela rendrait l'outil plus accessible, notamment pour des utilisateurs non techniques, et permettrait une meilleure visualisation des résultats : graphiques, tableaux, alertes visuelles.

Intégration avec la supervision

Une intégration directe avec **Zabbix** pourrait permettre d'automatiser et d'enrichir la collecte d'informations issues de diagnostic ou d'audit. Les résultats des modules pourraient être envoyés automatiquement, sous formes de **rapports** ou d'**alertes**, à Zabbix, ce qui renforcerait la corrélation entre les incidents techniques et les données métiers.

Automatisation planifiée et reporting

Il pourrait également être utile d'intégrer un système de **planification automatique** pour exécuter les diagnostics, sauvegardes ou audits de façon régulière et planifiée, sans besoin d'intervention manuelle. Cela permettrait aux équipes de gagner en efficacité et de se concentrer sur d'autres opérations.

Enfin, un module de **reporting automatisé** pourrait être intégré à l'outil, pour générer des **rapports en PDF** ou envoyer des alertes **par mail**, ce qui renforcerait la traçabilité et le suivi de l'état du système d'information.

Conclusion

La version finale de NTL-SysToolbox est fonctionnelle et complète, et s'exécute aussi bien sous Linux que sous Windows, conformément aux attentes de NTL.

Les trois modules - *Diagnostic*, *Sauvegarde* et *audit d'obsolescence* - fonctionnent de façon indépendante, tout en étant intégrés à un menu principal unique affiché au lancement du script de déploiement (.sh pour les distributions Linux, .bat pour les distributions Windows).

Les principaux objectifs ont été atteints :

- Le module **Diagnostic** assure une vérification complète de l'état des serveurs et des services essentiels (AD, DNS, MySQL, ressources système).
- Le module **Sauvegarde** permet les sauvegardes .sql et l'export de table (.csv)
- Le module **Audit** permet une visibilité claire sur la conformité et l'obsolescence de l'infrastructure grâce à des scans réseaux automatisés.

Ce projet présente ainsi plusieurs apports à Nord Transit Logistic, aussi bien sur le plan technique qu'organisationnel :

Automatisation

Les tâches d'exploitation (diagnostic, sauvegarde, audit) sont maintenant automatisées, ce qui réduit considérablement le temps d'intervention et les risques d'erreur humaine.

Sécurité

Les sauvegardes SQL et exports CSV assurent une traçabilité des données critiques. De plus, l'intégration de l'audit d'obsolescence permet d'anticiper les failles liées aux versions obsolètes.

Supervision et pilotage

L'outil permet une vision globale et centralisée de l'état du système d'information, facilitant le travail du service informatique et complétant la supervision technique.

Summary

The NTL-SysToolbox project was developed for Nord Transit Logistics (NTL), a logistics company based in northern France. Its goal was to design a lightweight and efficient command-line tool to simplify the daily administration of the company's IT infrastructure.

The company's information system relies on several Windows Server and Linux machines that provide essential services such as Active Directory, DNS, and the Warehouse Management System (WMS) based on a MySQL database.

To improve reliability and reduce manual work, the tool integrates three independent modules:

- Diagnostic module: checks network connectivity, DNS/AD availability, and server performance.
- Backup module: automates the complete or partial export of the WMS database in SQL and CSV formats, ensuring data integrity and traceability.
- Obsolescence audit module: scans the network to detect operating systems in use and compares them to official End-of-Life (EOL) data from software vendors.

The solution, written in Python, was designed to work on both Windows and Linux environments with minimal setup. Configuration parameters are stored in a .env file, allowing flexible deployment across different machines.

During testing, the tool successfully executed all modules, generating clear reports and outputs to support technical monitoring and decision-making.

The project demonstrates the advantages of automation and centralization in system administration: faster diagnostics, reliable backups, and proactive management of software lifecycles.

Future improvements could include a graphical interface, automated scheduling, or integration with supervision tools such as Zabbix to enhance monitoring and reporting capabilities.